Findora

(Documentation)

Devpost

Tech Builders Program Competition

Abdallah Fekry

16-4-2026

Table of Contents

1. Project Overview

Findora is an AI-powered product assistant designed to simplify the process of finding, comparing, and purchasing products. The system leverages advanced Artificial Intelligence techniques, including Large Language Models (LLMs) and agent-based workflows, to deliver personalized and data-driven recommendations.

Findora acts as a smart intermediary between the user and multiple data sources. It combines internal structured datasets with real-time external data from online marketplaces to provide accurate and up-to-date product recommendations.

The application allows users to:

- Search for products based on specific requirements (budget, brand, specifications)
- Compare multiple products intelligently
- Receive a top recommendation based on value and performance
- Access real-time purchase links from platforms like Amazon and eBay
- Interact through text and voice inputs

The system is implemented as a web application using Streamlit, providing an interactive and user-friendly interface.

---

2. Problem Statement

Modern online shopping presents several challenges:

- **Information Overload:** Users are exposed to thousands of products across multiple platforms.
- **Time Consumption:** Comparing specifications, prices, and reviews manually is time-consuming.
- **Lack of Decision Support:** Users often lack the technical expertise to evaluate products effectively.
- **Inconsistent Pricing:** Prices vary significantly across platforms, making it difficult to find the best deal.
- **Fragmented Experience:** Users must navigate multiple websites to complete a single purchase decision.

Findora addresses these challenges by automating the entire decision-making process. It reduces user effort by aggregating data, analyzing product specifications, and providing clear, actionable recommendations.

3. Project Structure

The project follows a modular structure divided into three main components:

3.1 Main Entry Point

- **app.py**
    - Initializes the application
    - Configures page settings
    - Manages navigation between pages
    - Initializes session state variables

3.2 Home Interface

- **home.py**
    - Serves as the landing page
    - Introduces the application and its features
    - Provides navigation to the chatbot interface
    - Displays services, FAQs, and user guidance

3.3 Chatbot System (Core Logic)

- **chat.py**
    - Handles user interaction
    - Implements AI agent logic
    - Defines tools for product search and selection
    - Integrates external APIs (Amazon, eBay)
    - Manages session state and chat history
    - Renders dynamic UI components for results

4. Technical Implementation and Methodologies

4.1 AI Agent Architecture

Uses an **agent-based architecture** powered by:

- **LangGraph (ReAct Agent Pattern)**
- **Google Gemini LLM**

The agent follows a structured workflow:

1. Understand user intent
2. Select the appropriate tool
3. Retrieve relevant data
4. Analyze and compare results
5. Generate a structured response

4.2 Internal Data Processing

The system uses structured datasets stored in CSV files:

- Mobile phones dataset
- Laptops dataset
- Cars dataset

Data processing is performed using **Pandas**, enabling:

- Filtering by budget and specifications
- Multi-condition queries
- Data aggregation and comparison

4.3 Tool-Based System

Custom tools are defined using the LangChain @tool decorator:

Core Tools:

- search_phone()
- search_laptop()
- search_car()

These tools:

- Accept structured parameters
- Filter datasets
- Return results for AI processing

Decision Tools:

- add_top_one_phone()
- add_top_one_laptop()
- add_top_one_car()

These tools store the final recommendation for UI visualization.

4.4 External API Integration

Findora integrates real-time data using:

Amazon Search (via SerpAPI)

- Retrieves product listings
- Extracts price, rating, and product details

eBay API

- Uses OAuth authentication
- Fetches live product listings and pricing

This hybrid approach ensures both **accuracy (internal data)** and **freshness (external data)**.

4.5 Chat System

The chat system supports:

- Text input
- Voice input (Speech-to-Text)
- Multi-turn conversations
- Context-aware responses

Conversation history is stored in st.session_state, enabling persistent interaction.

4.6 Speech Recognition

Speech input is handled using:

- speech_recognition library
- Google Speech API

The system supports:

- English
- Arabic

4.7 User Interface (UI)

The UI is built using **Streamlit** and includes:

- Chat interface with message bubbles
- Product comparison tables
- Pop-up product cards
- External product links
- Interactive feedback system

The interface is designed to be intuitive, responsive, and visually engaging.

4.8 Workflow Summary

1. User submits a query
2. AI agent processes the request
3. Internal database is searched
4. Top 3 products are selected
5. Best product is identified
6. External APIs fetch live deals
7. Results are displayed with UI components

5. Tools and Technologies

Programming Language

- Python

Frameworks and Libraries

- **Streamlit** → Web application interface
- **LangChain** → Tool and agent integration
- **LangGraph** → Agent workflow orchestration
- **Pandas** → Data manipulation and filtering
- **Requests** → API communication
- **SpeechRecognition** → Voice input processing

AI and Machine Learning

- **Google Gemini API** → Large Language Model for reasoning and decision-making

APIs and External Services

- **SerpAPI** → Amazon product search
- **eBay API** → Real-time product listings

Data Storage

- CSV files (structured datasets)
- Session-based storage using Streamlit

6. Future Improvements

6.1 System Architecture

- Refactor into a modular architecture (services, agents, UI layers)
- Introduce backend frameworks (FastAPI or Flask)

6.2 Performance Optimization

- Implement caching for API calls
- Optimize dataset queries

6.3 Advanced AI Features

- Multi-agent system (planner, retriever, comparator)
- Personalized recommendations based on user behavior

- Fine-tuned ranking algorithms

6.4 Data Expansion

- Add more product categories (e.g., tablets, accessories)
- Integrate additional marketplaces (e.g., Jumia, Noon)

6.5 User Experience

- Add authentication system
- Save user preferences
- Improve UI/UX design and animations

6.6 Deployment

- Deploy on cloud platforms (AWS, GCP, Azure)
- Add scalability support
- Implement monitoring and logging systems

Conclusion

Findora demonstrates the practical application of AI agents in solving real-world problems. By combining structured data, real-time APIs, and intelligent decision-making, the system provides a seamless and efficient shopping experience.

The project showcases strong capabilities in:

- AI system design
- Data processing
- API integration
- User interface development

With further improvements, Findora has the potential to evolve into a full-scale intelligent shopping platform.

Team Information

Contributors:

- Abdallah Fekry (working solo)